sending requests over the network, but in reality, the requests are kept entirely in memory.

Listing 36.7 shows how to use the Test Host package to create a simple integration test for the `StatusMiddleware`. First, create a `HostBuilder`, and call `ConfigureWebHost()` to define your application by adding middleware in the `Configure` method. This is equivalent to the `Startup.Configure()` method you would typically use to configure your application when using the generic host approach.

NOTE You can write a similar test using `WebApplicationBuilder`, but this sets up lots of extra defaults such as configuration, extra dependency injection (DI) services, and automatically added middleware, which can generally slow and add some confusion to simple tests. You can see an example of this approach in `StatusMiddlewareTestHostTests` in the source code for this book, but I recommend using the approach in listing 36.7, using `HostBuilder`, in most cases.

Call the `UseTestServer()` extension method in `ConfigureWebHost()`, which replaces the default Kestrel server with the `TestServer` from the Test Host package. The `TestServer` is the main component in the Test Host package, which makes all the magic possible. After configuring the `HostBuilder`, call `StartAsync()` to build and start the test application. You can then create an `HttpClient` using the extension method `GetTestClient()`. This returns an `HttpClient` configured to make in-memory requests to the `TestServer`, as shown in the following listing.